Methoden-Referenzen

Carsten Gips (HSBI)

TL;DR
Seit Java8 können **Referenzen auf Methoden** statt anonymer Klassen eingesetzt werden (**funktionales Interface nötig**).
Dabei gibt es drei mögliche Formen:

- Form 1: Referenz auf eine statische Methode: `ClassName::staticMethodName` (wird verwendet wie `(args)-> ClassName.staticMethodName(args)`)
- Form 2: Referenz auf eine Instanz-Methode eines Objekts: `objectref::instanceMethodName` (wird verwendet wie `(args)-> objectref.instanceMethodName(args)`)
- Form 3: Referenz auf eine Instanz-Methode eines Typs: `ClassName::instanceMethodName` (wird verwendet wie `(o1, args)-> o1.instanceMethodName(args)`)

Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode. Die Methoden-Referenz muss von der Syntax her dieser einen abstrakten Methode entsprechen (bei der dritten Form wird die Methode auf dem ersten Parameter aufgerufen).
Videos

- VL Methoden-Referenzen
- Demo Referenz auf statische Methode
- Demo Referenz auf Instanz-Methode (Objekt)
- Demo Referenz auf Instanz-Methode (Typ)

---

1 Beispiel: Sortierung einer Liste

```java
List<Studi> sl = new ArrayList<Studi>();

// Anonyme innere Klasse
Collections.sort(sl, new Comparator<Studi>() {
    @Override public int compare(Studi o1, Studi o2) {
        return Studi.cmpCpsClass(o1, o2);
    }
});


// Lambda-Ausdruck
Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));

// Methoden-Referenz
Collections.sort(sl, Studi::cmpCpsClass);
```

1.1 Anmerkung

Für das obige Beispiel wird davon ausgegangen, dass in der Klasse `Studi` eine statische Methode `cmpCpsClass()` existiert:

```java
public static int cmpCpsClass(Studi s1, Studi s2) {
    return s1.getCps() - s2.getCps();
}
```

Wenn man im Lambda-Ausdruck nur Methoden der eigenen Klasse aufruft, kann man das auch direkt per *Methoden-Referenz* abkürzen!

- Erinnerung: `Comparator<T>` ist ein funktionales Interface
- Instanzen können wie üblich durch Ableiten bzw. anonyme Klassen erzeugt werden
- Alternativ kann seit Java8 auch ein passender Lambda-Ausdruck verwendet werden
- Ab Java8: Referenzen auf passende Methoden (Signatur!) können ein funktionales Interface "implementieren"
  - Die statische Methode `static int cmpCpsClass(Studi s1, Studi s2)` hat die selbe Signatur wie `int compare(Studi s1, Studi s2)` aus `Comparator<Studi>`
  - Kann deshalb wie eine Instanz von `Comparator<Studi>` genutzt werden
  - Name der Methode spielt dabei keine Rolle

2 Überblick: Arten von Methoden-Referenzen

1. Referenz auf eine statische Methode
   - Form: `ClassName::staticMethodName`
   - Wirkung: Aufruf mit `(args)-> ClassName.staticMethodName(args)`
2. Referenz auf Instanz-Methode eines bestimmten Objekts
   - Form: `objectref::instanceMethodName`
   - Wirkung: Aufruf mit `(args)-> objectref.instanceMethodName(args)`
3. Referenz auf Instanz-Methode eines bestimmten Typs
   - Form: `ClassName::instanceMethodName`
   - Wirkung: Aufruf mit `(arg0, rest)-> arg0.instanceMethodName(rest)` (`arg0` ist vom Typ `ClassName`)

Anmerkung: Analog zur Referenz auf eine statische Methode gibt es noch die Form der Referenz auf einen Konstruktor: `ClassName::new`. Für Referenzen auf Konstruktoren mit mehr als 2 Parametern muss ein eigenes passendes funktionales Interface mit entsprechend vielen Parametern definiert werden …

3 Methoden-Referenz 1: Referenz auf statische Methode

```java
public class Studi {
    public static int cmpCpsClass(Studi s1, Studi s2) {
        return s1.getCredits() - s2.getCredits();
    }
```

```java

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();

        // Referenz auf statische Methode
        Collections.sort(sl, Studi::cmpCpsClass);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> Studi.cmpCpsClass(o1, o2));
    }
}
```

Demo: methodreferences.DemoStaticMethodReference (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/modern-java/src/methodreferences/DemoStaticMethodReference.java)

`Collections.sort()` erwartet in diesem Szenario als zweiten Parameter eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **statische Methode `cmpCpsClass` der Klasse `Studi`** hat die **selbe Signatur** und wird deshalb von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich **`Studi.cmpCpsClass(o1, o2)`** aufgerufen.

4 Methoden-Referenz 2: Referenz auf Instanz-Methode (Objekt)

```java
public class Studi {
    public int cmpCpsInstance(Studi s1, Studi s2) {
        return s1.getCredits() - s2.getCredits();
    }

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();
        Studi holger = new Studi("Holger", 42);

        // Referenz auf Instanz-Methode eines Objekts
        Collections.sort(sl, holger::cmpCpsInstance);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> holger.cmpCpsInstance(o1, o2));
    }
}
```

Demo: methodreferences.DemoInstanceMethodReferenceObject (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/modern-java/src/methodreferences/DemoInstanceMethodReferenceObject.java)

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode `int compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Objekts `holger`** hat die selbe

Signatur und wird entsprechend von `Collections.sort()` genauso genutzt wie die eigentlich erwartete Methode `Comparator<Studi>#compare(Studi o1, Studi o2)`, d.h. statt `compare(o1, o2)` wird nun für jeden Vergleich **`holger.cmpCpsInstance(o1, o2)`** aufgerufen.

5 Methoden-Referenz 3: Referenz auf Instanz-Methode (Typ)

```
public class Studi {
    public int cmpCpsInstance(Studi studi) {
        return this.getCredits() - studi.getCredits();
    }

    public static void main(String... args) {
        List<Studi> sl = new ArrayList<Studi>();

        // Referenz auf Instanz-Methode eines Typs
        Collections.sort(sl, Studi::cmpCpsInstance);

        // Entsprechender Lambda-Ausdruck
        Collections.sort(sl, (o1, o2) -> o1.cmpCpsInstance(o2));
    }
}
```

Demo: methodreferences.DemoInstanceMethodReferenceType (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/modern-java/src/methodreferences/DemoInstanceMethodReferenceType.java)

`Collections.sort()` erwartet in diesem Szenario als zweites Argument wieder eine Instanz von `Comparator<Studi>` mit einer Methode **`int`** `compare(Studi o1, Studi o2)`.

Die übergebene Referenz auf die **Instanz-Methode `cmpCpsInstance` des Typs `Studi`** hat die Signatur **`int`** `cmpCpsInstance(Studi studi)` und wird von `Collections.sort()` so genutzt: Statt `compare(o1, o2)` wird nun für jeden Vergleich **`o1.cmpCpsInstance(o2)`** aufgerufen.

6 Ausblick: Threads

Erinnerung an bzw. Vorgriff auf "Threads: Intro":

```
public interface Runnable {
    void run();
}
```

Damit lassen sich Threads auf verschiedene Arten erzeugen:

```
public class ThreadStarter {
    public static void wuppie() { System.out.println("wuppie(): wuppie"); }
}


Thread t1 = new Thread(new Runnable() {
```

```
7       public void run() {
8           System.out.println("t1: wuppie");
9       }
10  });
11
12  Thread t2 = new Thread(() -> System.out.println("t2: wuppie"));
13
14  Thread t3 = new Thread(ThreadStarter::wuppie);
```

Beispiel: methodreferences.ThreadStarter (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/modern-java/src/methodreferences/ThreadStarter.java)

7 Ausblick: Datenstrukturen als Streams

Erinnerung an bzw. Vorgriff auf "Stream-API":

```
1  class X {
2      public static boolean gtFour(int x) { return (x > 4) ? true : false; }
3  }
4
5  List<String> words = Arrays.asList("Java8", "Lambdas", "PM",
6          "Dungeon", "libGDX", "Hello", "World", "Wuppie");
7
8  List<Integer> wordLengths = words.stream()
9          .map(String::length)
10         .filter(X::gtFour)
11         .sorted()
12         .collect(toList());
```

Beispiel: methodreferences.CollectionStreams (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/modern-java/src/methodreferences/CollectionStreams.java)

- Collections können als Datenstrom betrachtet werden: `stream()`
  - Iteration über die Collection, analog zu externer Iteration mit `foreach`
- Daten aus dem Strom filtern: `filter`, braucht Prädikat
- Auf alle Daten eine Funktion anwenden: `map`
- Daten im Strom sortieren: `sort` (auch mit Comparator)
- Daten wieder einsammeln mit `collect`

=> Typische Elemente **funktionaler Programmierung**

=> Verweis auf Wahlfach "Spezielle Methoden der Programmierung"

8 Wrap-Up

Seit Java8: **Methoden-Referenzen** statt anonymer Klassen (**funktionales Interface nötig**)

- Drei mögliche Formen:
  - Form 1: Referenz auf statische Methode: `ClassName::staticMethodName` (verwendet wie `(args)-> ClassName.staticMethodName(args)`)
  - Form 2: Referenz auf Instanz-Methode eines Objekts: `objectref::instanceMethodName` (verwendet wie `(args)-> objectref.instanceMethodName(args)`)
  - Form 3: Referenz auf Instanz-Methode eines Typs: `ClassName::instanceMethodName` (verwendet wie `(o1, args)-> o1.instanceMethodName(args)`)
- Im jeweiligen Kontext muss ein passendes funktionales Interface verwendet werden (d.h. ein Interface mit **genau** einer abstrakten Methode)

9 Zum Nachlesen

- Oracle Corporation (2022)
- Urma, Fusco, und Mycroft (2014, Kap. 3)

Lernziele

- k2: Funktionales Interfaces (Definition)
- k3: Einsatz von Methoden-Referenzen

Quizzes

- Quiz Methoden-Referenzen (ILIAS)

Challenges

In den Vorgaben finden Sie die Klassen `Student` und `StudentSort` mit vorgefertigten Methoden zu den Teilaufgaben sowie eine Testsuite `SortTest` mit einzelnen Testfälllen zu den Teilaufgaben, mit der Ihre Implementierung aufgerufen und getestet wird.
Ziel dieser Aufgabe ist es, eine Liste von Studierenden mithilfe verschiedener syntaktischer Strukturen (Lambda-Ausdrücke, Methoden-Referenzen) zu sortieren. Dabei soll bei allen Teilaufgaben die Methode java.util.List#sort für das eigentliche Sortieren verwendet werden.

1. Erweitern Sie die Klasse `Student` um eine *statische* Methode, die zwei `Student`-Objekte anhand des Alters miteinander vergleicht. Die Methode soll die Signatur `static int compareByAge(Student a, Student b)` besitzen und die folgenden Werte zurückliefern:
   - a > b -> -1
   - a < b -> 1
   - a == b -> 0

   Verwenden Sie die neue statische Methode `compareByAge` zum Sortieren der Liste in `sort_2b()`. Nutzen Sie dabei eine **Methodenreferenz**.
2. Erweitern Sie die Klasse `Student` um eine Instanz-Methode, die das `Student`-Objekt mit einem anderen (als Parameter übergebenen) `Student`-Objekt vergleicht. Die Methode soll die Signatur `int compareByName (Student other)` besitzen und die folgenden Werte zurückliefern:
   - self > other -> -1
   - self < other -> 1
   - self == other -> 0

   Verwenden Sie die neue Methode `compareByName` zum Sortieren der Liste in `sort_3b()`. Nutzen Sie dabei eine **Methodenreferenz**.

Quellen

Oracle Corporation. 2022. „The Java Tutorials“. 2022. https://docs.oracle.com/javase/tutorial/.
Urma, R.-G., M. Fusco, und A. Mycroft. 2014. *Java 8 in Action: Lambdas, Streams, and Functional-Style Programming*. Manning Publications.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)